

Trainer Guide: Spring Boot Session 4 - Microservices with Spring Cloud

A slide-by-slide read-aloud script for [springboot-session4-theory.html](#). Everything on screen, explained in plain spoken language.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [springboot-session4-theory.html](#) (22 slides)

Each slide has an ON SCREEN reminder and a SAY section you can read out word-for-word. EMPHASISE and DELIVERY TIP notes are for you only - do not read those aloud.

Slide 1 - Title: Microservices with Spring Cloud

ON SCREEN

The blue cover slide marking the final session: Service Registry, API Gateway & Load Balancing.

SAY (read aloud)

Welcome to Session 4 - the final session in our Spring Boot series. Session 3 ended with the idea of microservices. Today we build the infrastructure that runs them. By the end you'll have four Spring Boot apps running at once - a registry, two microservices, and a gateway - all talking to each other and load-balancing traffic. This is the most infrastructure-heavy session, so the theory is front-loaded, and every concept maps directly to something you'll build in the lab.

Slide 2 - Agenda

ON SCREEN

Six agenda items: monolith vs microservices recap, the design patterns, Spring Cloud, architecture flow, Eureka, API Gateway.

SAY (read aloud)

Here's the plan. A sixty-second anchor on monolith versus microservices. The five microservices design patterns. Spring Cloud - the toolbox that implements them. The full architecture flow, which is the mental model for the whole session. Then Eureka, the service registry, with its server and client. And finally the API Gateway, with routing and round-robin load balancing. Keep slide 14, the architecture flow, in your mind throughout the lab - it's the map for everything we build.

Slide 3 - Session 3 recap

ON SCREEN

Three click-to-reveal flash cards: what a DTO does, @ResponseStatus, and what Spring Security adds.

SAY (read aloud)

Three quick warm-ups. Question one: what is a DTO and why use it? A plain Java class that carries only the data the client needs - it hides sensitive fields, shapes responses, and decouples the API from the database schema. Question two: what does @ResponseStatus do on a custom exception? It tells Spring which HTTP status to return when that exception is thrown - so ResourceNotFoundException with NOT underscore FOUND yields a clean 404. Question three: what does Spring Security do the moment its dependency is added? It locks down every endpoint - a login page appears, a random password is printed in the console, and the default username is 'user' - all before you write any code.

Slide 4 - Monolith vs microservices, revisited

ON SCREEN

A two-column recap: monolith versus microservices, with a key box on today's framing.

SAY (read aloud)

A quick sixty-second anchor. Last session we discussed why microservices exist; today we build the infrastructure that runs them. Quick contrast. A monolith is one deployable unit, one shared database, where one failure can mean a full outage, and scaling means duplicating the whole app. Microservices are independent services, each with its own database, with failure isolation - one down, others up - and you scale only the service that needs it.

EMPHASISE

Set today's framing: microservices give independence and scalability, but they introduce a new question - how do all these separate, independently-deployed services find and talk to each other? That's the problem the rest of the session solves.

Slide 5 - Horizontal vs vertical scaling, revisited

ON SCREEN

Two columns - vertical and horizontal scaling - and a warning box introducing the two new problems.

SAY (read aloud)

A reminder of the two scaling styles. Vertical - scale up - means one machine with more RAM and CPU; simple, but with a hard ceiling and expense. Horizontal - scale out - means more machines running the same service; no real ceiling, cost-effective, fault-tolerant. Now here's the catch that sets up the whole session. Horizontal scaling means you'll have multiple instances of the same service - for example, Student MS on ports 8080, 8082, and 8083. That creates two brand-new problems: how does the client know all the addresses, when instances come and go on dynamic ports? And how does traffic get distributed evenly across them?

EMPHASISE

Name the solutions: those two problems are solved by the next two patterns - a Service Registry and an API Gateway. Everything today flows from this slide.

Slide 6 - The five microservices design patterns

ON SCREEN

Five cards: Config Server, Service Registry, API Gateway, Distributed Tracing, Circuit Breaker.

SAY (read aloud)

Five patterns come up in every real microservices system, each solving one specific pain. We'll build the first two today and cover the rest conceptually. Config Server: ten services with ten properties files becomes one central, Git-backed config. Service Registry: dynamic ports you can't hardcode becomes a live catalog of what's running. API Gateway: clients shouldn't know internal URLs, so you give them one single entry point. Distributed Tracing: a request spanning five services becomes end-to-end tracking so you can see what's slow. And Circuit Breaker: when Service B is down and A keeps calling it, the breaker detects the failure, stops calling, and returns a fallback.

EMPHASISE

Be explicit about scope: today we BUILD the Service Registry with Eureka and the API Gateway. Config Server, Distributed Tracing, and Circuit Breaker are concept-only - knowing they exist and what each solves is what matters.

Slide 7 - Config Server

ON SCREEN

A diagram: a Git repo served by Spring Cloud Config Server, read by Services A, B, and C.

SAY (read aloud)

Pattern one, concept only. Imagine ten microservices, each with its own properties file. You need to change the database URL - now you're editing ten files and redeploying ten services. The Config Server fixes this: every service pulls its configuration from one central, Git-backed source at startup. Follow the diagram: a Git repository holds all the config, version-controlled. Spring Cloud Config Server serves it. And Services A, B, and C all read from it as they start. When would you use it? When you have environment-specific configs - dev, test, prod - across many services, and want a single audited place to manage them. We're not building it today, but you'll meet it in real projects.

Slide 8 - Service Registry (Eureka), how it works

ON SCREEN

A four-step flow: Eureka starts, Student MS registers, Customer MS registers, the Gateway asks Eureka. A key box on heartbeats.

SAY (read aloud)

Pattern two, and we're building this today. A Service Registry is a live phonebook of every running service - its name, address, and port. Netflix Eureka is the registry we'll use. Services register themselves on startup and prove they're alive with a heartbeat. Follow the four steps. One: Eureka starts on port 8761 - the phonebook itself, empty at first. Two: Student MS starts and tells Eureka 'I'm STUDENT-MS at port 8080', sending a heartbeat about every thirty seconds. Three: Customer MS registers the same way. Four: the Gateway asks Eureka 'where is STUDENT-MS?' and Eureka returns the live list of instances.

EMPHASISE

Highlight the self-healing angle in the key box: registered services ping Eureka regularly. If a service crashes and the heartbeats stop, Eureka removes it from the registry after a grace period - so the gateway stops routing to a dead instance automatically. No manual cleanup.

Slide 9 - API Gateway, what it does

ON SCREEN

Three cards - routing, load balancing, single entry point - and a sample gateway route in properties.

SAY (read aloud)

Pattern three, also building today. The API Gateway is the single front door to your whole system. Clients talk to one URL; the gateway figures out the rest. It has three core jobs. Routing: forward slash-students-api to Student MS and slash-customers-api to Customer MS, based on the URL. Load balancing: spread traffic across multiple instances of a service so none is overwhelmed. And single entry point: clients only ever know one URL - the gateway, on port 9393 - and the internal services stay hidden. Look at the sample route: it has an id, a uri of lb colon slash slash STUDENT-MS, and a predicate matching the path. We'll write exactly this in the lab. Secondary jobs, mention-only: authentication, rate limiting, and logging are all natural to centralise here.

Slide 10 - Load balancing: Round Robin

ON SCREEN

A visual of four requests cycling across instances 8080, 8082, 8083, then back to 8080. A key box on the lb:// prefix.

SAY (read aloud)

When several instances of a service are running, the gateway needs a rule to pick one per request. The simplest, and the default, is Round Robin: just cycle through the instances in order, then loop back to the start. Watch the visual: request one goes to 8080, request two to 8082, request three to 8083, and request four loops back to 8080. Traffic spreads evenly with no configuration.

EMPHASISE

Give them the magic word: the lb-colon-slash-slash prefix in the gateway route is what switches this on. lb means load balancer. It tells Spring Cloud - don't use a hardcoded address; ask Eureka for all live instances and round-robin across them automatically. They'll watch this happen live in the lab.

Slide 11 - Distributed Tracing

ON SCREEN

A request flowing through Gateway, Order MS, Inventory MS, Payment MS, and a Zipkin-style timing breakdown.

SAY (read aloud)

Pattern four, concept only. When one request hops through five services and the whole thing is slow, which service caused it? Distributed tracing answers that. Every request gets a unique trace ID that's passed along the chain; each service logs with that ID, and a tool like Zipkin stitches them into one timeline. Follow the flow: the client hits the Gateway, which calls Order MS, which calls Inventory MS, which calls Payment MS. Each box adds the same trace ID to its logs. The timing breakdown then shows exactly where the time went - in the example, Inventory MS is the slow one at over two seconds. In the Spring world this is done with Micrometer Tracing and Zipkin. Conceptual only today, but invaluable the moment you have more than two services.

Slide 12 - Circuit Breaker

ON SCREEN

A three-state diagram: CLOSED, OPEN, HALF-OPEN, and the transitions between them.

SAY (read aloud)

Pattern five, concept only. If Service B is down, Service A shouldn't keep hammering it - that wastes resources and can cascade into a system-wide failure. A circuit breaker wraps the call and watches for failures, just like an electrical breaker that trips to prevent damage. It has three states. CLOSED is normal - requests flow through and failures are counted. OPEN means too many failures, so the breaker trips - requests are rejected instantly and a fallback is returned, no more hammering. HALF-OPEN happens after a timeout: it lets one trial request through; if it succeeds, back to CLOSED; if it fails, back to OPEN. The analogy is a household breaker - it trips to prevent damage, then lets you check if it's safe to resume. In Spring this is Resilience4j. Not building it today.

Slide 13 - Spring Cloud, the toolbox

ON SCREEN

A table mapping each pattern to its Spring Cloud tool, with a 'today?' column.

SAY (read aloud)

Now let's connect patterns to tools. Spring Cloud isn't one library - it's a collection of tools that implement these patterns on top of Spring Boot. You pick the pieces you need. Read the table. Service Registry maps to Netflix Eureka - building today. API Gateway maps to Spring Cloud Gateway - building today. Load Balancing maps to Spring Cloud LoadBalancer, which we use via the lb prefix. Config Server maps to Spring Cloud Config - concept. Circuit Breaker maps to Resilience4j - concept. And Distributed Tracing maps to Micrometer plus Zipkin - concept. The point: you don't use every tool in every project - you assemble the toolbox your system needs. Today it's Eureka plus Gateway plus LoadBalancer, and that trio alone is a genuine microservices skeleton.

Part 2 - The Architecture & What We Build

Slide 14 - Full architecture flow

ON SCREEN

The centrepiece diagram: client to API Gateway (9393), which queries Eureka (8761), then routes to Student MS instances and Customer MS.

SAY (read aloud)

This is the centrepiece - the mental model for the whole session. Let's trace a single request top to bottom, and we'll refer back to this diagram throughout the lab. A React client or Postman knows only one URL. It sends a request to port 9393, slash-students-api. The API Gateway, the single entry point, receives it. The gateway asks the Eureka Server, on port 8761, 'where is STUDENT-MS?' and Eureka returns the list of live instances. The gateway then picks one via round robin and forwards the request - to Student MS on 8080, 8082, or 8083. Customer MS, on 8081, works the same way.

EMPHASISE

Read it as a sentence they can repeat: the client hits the gateway; the gateway asks Eureka which instances are alive; Eureka returns the list; the gateway round-robins to one and forwards the request; the response travels back. The client never knew there were three Student MS instances.

Slide 15 - What we're building today

ON SCREEN

Four project cards: Eureka Server, Student MS, Customer MS, API Gateway, each with its port and key dependency.

SAY (read aloud)

Here's the build map - four independent Spring Boot projects, each with one key dependency and, where needed, one key annotation. Eureka Server on port 8761, with the Eureka Server dependency and the `@EnableEurekaServer` annotation. Student MS on 8080, 8082, and 8083, with Web plus Eureka Client, registered as STUDENT-MS. Customer MS on 8081, same dependencies, registered as CUSTOMER-MS. And the API Gateway on 9393, with Gateway plus Eureka Client, configured with routes only - no controller.

DELIVERY TIP

Warn them before coding: they'll run four, then six, JVMs at once - that needs some RAM. If a machine struggles, it's fine to stop the extra Student MS instances after seeing them register. The lab sheet has the exact ports and steps.

Slide 16 - Eureka Server setup

ON SCREEN

The main class with `@EnableEurekaServer`, and the four key `application.properties` lines.

SAY (read aloud)

The Eureka Server is almost all configuration - one annotation and four properties. On the main class, `@EnableEurekaServer` turns this app into the registry. In `application.properties`: `server.port` is 8761, the standard Eureka port clients expect. Then two important lines - `register-with-eureka` is false, and `fetch-registry` is false. Why those two falses? Eureka components are clients by default. For the server we switch both off: it has no reason to register with itself, and no reason to download a registry from elsewhere - it IS the registry. Leave them out and you'll see noisy startup errors as it tries to talk to a registry that doesn't exist.

Slide 17 - Eureka Client setup

ON SCREEN

The three key `application.properties` lines for a microservice, and a key box on the service name.

SAY (read aloud)

Any microservice that wants to join the registry just needs the Eureka Client dependency and three properties. No annotation is required in modern Spring Cloud - it auto-detects the client from the classpath; older versions used `@EnableEurekaClient`. The three lines: `server.port` - 8080, which we'll override at runtime for more instances. `spring.application.name` - STUDENT-MS, the service ID. And `eureka.client.service-url.defaultZone`, pointing at the registry at `localhost:8761/eureka`.

EMPHASISE

Stress the single most important line: `spring.application.name`. This exact name is what Eureka lists the service under, and what the Gateway uses to route, with `lb://STUDENT-MS`. Get it wrong or mismatched and the gateway route silently fails to resolve. Convention is uppercase service IDs.

Slide 18 - API Gateway: manual routes vs discovery locator

ON SCREEN

Two panels: manual routing config versus discovery-locator config, with pros and cons.

SAY (read aloud)

There are two ways to tell the gateway how to route, and you'll try both in the lab. Manual routing, on the left: you define each route with an id, a uri of `lb://` plus the service name, and a path predicate. You get clean custom URLs like `/students-api`, and more control - but more config to write per service. Discovery locator, on the right: one block of two lines auto-creates routes for every service registered in Eureka. Zero per-service config - but the service name appears in the URL, like `/student-ms`.

EMPHASISE

Give the recommendation: use manual routes for production-grade, clean public APIs where you control the URL shape; use discovery locator for quick prototyping. The trade-off is URL cleanliness versus zero configuration.

Slide 19 - Running multiple instances

ON SCREEN

Two columns - IntelliJ and Eclipse steps - and a mock Eureka dashboard showing STUDENT-MS with three instances.

SAY (read aloud)

To see load balancing, you need more than one instance of Student MS. You don't create new projects - you run the same project again on a different port using a VM argument. In IntelliJ: Edit Configurations, copy the Student MS config, and add the VM option dash-D-server-dot-port equals 8082; repeat for 8083. In Eclipse or STS: Run Configurations, a Maven build with goal spring-boot-run, and the same VM argument. Then refresh the Eureka dashboard - as the mock shows, STUDENT-MS now lists three instances, on 8080, 8082, and 8083.

EMPHASISE

Frame what they just did: they're simulating horizontal scaling on one laptop. In production these would be separate machines or containers - but to Eureka and the gateway, three instances on three ports look exactly like three instances on three servers.

Recap & Close

Slide 20 - The full journey, Sessions 1 to 4

ON SCREEN

A vertical timeline: Session 1 single controller, Session 2 REST API, Session 3 layers, Session 4 microservices.

SAY (read aloud)

Step back and look how far this has come - four sessions, one continuous evolution. Session 1: a single Spring Boot app with one controller - the basics of the framework. Session 2: a full REST API with CRUD, JSON, and status codes. Session 3: layered and production-shaped, with DTOs, service and repository layers, custom exceptions, and a database. And Session 4: a microservices architecture, with a registry, a gateway, multiple instances, and load balancing. Each concept built on the last. You've gone from a Hello World endpoint to a microservices architecture in four sessions - and what you can build now is a genuine, production-grade microservices skeleton.

Slide 21 - What's next: the industry roadmap

ON SCREEN

Six cards: Config Server, JWT security, Docker, Kubernetes, CI/CD, tracing with Zipkin.

SAY (read aloud)

What you built today is the skeleton; a real production project layers more on top. Framed as context, not pressure: a Config Server for centralised, Git-backed configuration. JWT security enforced at the gateway and shared across services. Docker to package each service into a container that runs the same everywhere. Kubernetes to orchestrate, scale, and self-heal those containers. CI/CD pipelines for automated build, test, and deploy per service. And distributed tracing with Zipkin for end-to-end visibility. You now have the foundation every one of these builds on - that's the real win of these four sessions.

Slide 22 - Closing

ON SCREEN

The closing blue slide marking the end of the series: 'Let's build it.'

SAY (read aloud)

And that brings us to the end of the theory, and the end of the series. The last sentence to leave you with: you started with a single Hello World controller, and you're finishing by standing up a microservices system with a registry, a gateway, and load balancing. Now let's open the lab - we'll start Eureka, register two microservices, scale one to three instances, and route everything through a gateway with live load balancing. Let's build it. And thank you, all of you, for completing the Spring Boot series.

DELIVERY TIP

During the lab, having Eureka pre-running on your machine as a safety net keeps the live demo smooth while associates catch up. Leave the slide 14 architecture diagram visible throughout the hands-on section.